12 August 2026 23:32

==========
DAY 02 - CLOUD COMPUTING BASICS
==========

1. WHAT IS CLOUD COMPUTING?

Cloud computing = renting computing resources (servers, storage, databases, networking, software) over the internet instead of buying and maintaining physical hardware yourself.

Think of it like a "computing provider" — similar to how GoDaddy is a web hosting provider, cloud providers rent you infrastructure/platforms/software on demand.

Major Cloud Providers & their portals:
- AWS : aws.amazon.com
- Azure : portal.azure.com
- GCP : console.cloud.google.com

2. TYPES OF CLOUD
---
a) PUBLIC CLOUD
- Resources owned & operated by a third-party provider, shared across multiple customers over the internet.
- Examples: AWS, Azure, GCP

b) PRIVATE CLOUD
- Cloud infrastructure used exclusively by one organization, usually hosted on-prem or in a dedicated data center. Gives more control/security.
- Examples: IBM RedHat OpenStack, VMware vCloud Director

c) HYBRID CLOUD
- A mix of public + private/on-prem, connected together so workloads can move between them.
- Examples: Azure Arc, AWS Outpost, GCP Anthos

[Hybrid cloud architecture explained]:
Onprem datacenter hardware (your own physical servers) connects to the Azure Cloud via an Interface/Software layer. This lets you manage on-prem + cloud resources from one place (single pane of glass).

Diagram logic:
Users -> Interface|Software -> Azure Cloud ->
On-prem datacenter hardware

(Azure Arc = Microsoft's tool to manage on-prem servers as if they were Azure resources)
(AWS Outpost = AWS hardware installed physically in your own data center, managed via AWS console)
(GCP Anthos = Google's platform to manage apps across on-prem, GCP, and even other clouds)

3. CLOUD SERVICE MODELS (IaaS / PaaS / SaaS)

These models describe HOW MUCH you manage yourself vs how much the provider manages for you.

Layers (bottom to top): Infra -> Platform -> Software

Layer	What it includes
Software	SQL files, web application code
Platform	Database, Web service, File service
Infra	CPU, Memory, Disk, Network, IP, LB (Load Balancer - distributes traffic across multiple servers)

a) IaaS (Infrastructure as a Service)
- Provider gives you: Infra (compute, storage, network)
- You manage: Platform + Software (OS, database, apps, code — everything above the hardware)
- Most control, most responsibility.
- Example: Azure VMs, AWS EC2

b) PaaS (Platform as a Service)
- Provider gives you: Infra + Platform (servers, DB engine, web service runtime already set up)
- You manage: only Software (your application code, SQL files)
- Example: Azure App Service, AWS Elastic Beanstalk

c) SaaS (Software as a Service)
- Provider gives you: Infra + Platform + Software — everything, fully managed. You just use the app.
- You manage: nothing (maybe just user settings/data)
- Examples: O365 Exchange, MS Teams, Azure AD, Azure Monitoring, Azure DevOps

[Quick memory trick]:
IaaS = you build the house on rented land
PaaS = you get a furnished house, you just decorate
SaaS = you get a hotel room, fully serviced

4. MANAGED vs UNMANAGED SERVICES

- UNMANAGED SERVICE: You (the customer) are responsible for managing everything — OS updates, patching, scaling, backups, security, etc.

- MANAGED SERVICE: The provider handles the operational work (patching, scaling, backups, uptime) for you. You get limited or no direct access to the underlying system.

  [Example]: An Azure VM (IaaS) is unmanaged — you patch the OS yourself. Azure SQL Database (PaaS) is managed — Microsoft handles patching, backups, and high availability for you.

5. ORCHESTRATION CONCEPT (Cloud Management Layer)

[Orchestration = the automated coordination of multiple systems/services so they work together without manual step-by-step intervention]

General orchestration hierarchy seen in both VM platforms and cloud portals:

```
  Orchestration
      |
  Scheduling   [Scheduling = deciding WHEN and WHERE a
               task/VM/container should run]
      |
  Controller   [Controller = component that continuously
               checks the actual state vs desired state
               and takes action to match them]
      |
  Manager      [Manager = coordinates resources and
               delegates tasks to the right node/host]
      |
  Database     [Stores the state/config info of the
               whole system]
```

Example 1 — Azure Cloud Orchestration:
  Users --> https://portal.azure.com -->
  [USA DC | India DC | Germany DC]
  (DC = Data Center)

  [Selfservice Multitenancy explained]:
  Multiple independent users/customers ("tenants") can log into the same portal and provision their own resources independently, without seeing each other's data — this is "self-service" because users don't need to ask an admin, they provision it themselves.

Example 2 — VMware Orchestration (on-prem equivalent):
  Users --> VCENTER --> [VMware ESXi Host 1 | Host 2 |

Host 3]

[VMware ESXi = a hypervisor, software that lets one physical server run multiple virtual machines (VMs)]
[vCenter = the central management tool that controls multiple ESXi hosts and their VMs]

This is the SAME pattern as Azure's portal -> data centers structure — just on-premises instead of cloud.
This overall concept is called:
[VM Orchestration = managing multiple virtual machines automatically]
[Container Orchestration = managing multiple containers automatically, e.g., Kubernetes]

==========
DAY 03 - ACCESS INTERFACES, RESOURCE HIERARCHY & BILLING
==========

1. WAYS TO INTERACT WITH A CLOUD/PLATFORM/APPLICATION

Every layer of the tech stack (OS, platform, application) usually offers 3 ways to interact with it:

- GUI/UI : Graphical/User Interface — click-based, e.g. a website or dashboard (used by humans)
- CLI : Command Line Interface — typed commands, e.g. "az", "kubectl", "aws" (used by humans, faster for repetitive tasks)
- API : Application Programming Interface — used for software-to-software communication (used by OTHER APPLICATIONS, not humans directly)

[Example from diagram - IRCTC ticket booking system]:

Layer	Software Used	Interface
Platform	Kubernetes	UI/GUI, CLI
OS software	Linux	UI/GUI, CLI
Application Server Software	IRCTC	UI/GUI, API
Platform	Cloud	UI/GUI, CLI

IRCTC talks to PhonePay and PayTM using API — this is called "Software to Software communication interface" because no human is manually clicking buttons; IRCTC's backend code directly calls PhonePay/PayTM's backend code to process payment.

2. INTERACTIVE vs NON-INTERACTIVE ACCESS (Very Important)

When accessing any cloud provider (Azure/AWS/GCP), access

happens through 2 categories of accounts:

a) USER ACCOUNT (Interactive)
- Used by an actual human (you)
- Human --> User account --> Interactive access --> GUI or CLI

[Interactive = requires a human to type/click in real time, e.g. logging into portal.azure.com yourself]

b) SERVICE ACCOUNT (Non-interactive)
- Used by an application/automation tool, not a human
- Application (e.g. Terraform) --> Service Account --> Non-interactive access --> API

[Non-interactive = no human sits and clicks; a script or tool authenticates and runs on its own using stored credentials]

[Terraform = an Infrastructure-as-Code tool that creates/ manages cloud resources automatically using API calls, instead of a human clicking in the portal]

[Service Account = a special account meant for apps/bots to log in programmatically, not for humans]

Interface comparison across providers:

Interface	Azure	GCP	AWS
CLI	az	gcloud	aws
GUI	Portal	Portal	Portal
API	API URL	API URL	API URL

[az / gcloud / aws = the command-line tool names you install on your laptop to control each cloud from a terminal]

3. AZURE CLOUD PORTAL - SERVICES vs RESOURCES

- SERVICE = the general category/type of offering (e.g. Virtual Machine, Disk, Load Balancer)
- RESOURCE = the actual instance you created of that service (e.g. myvm, mydisk, mylb)

[Analogy]: "Car" is a service/category. "My red Honda City with number plate DL01AB1234" is the resource — the specific real thing you own.

4. RESOURCE HIERARCHY MANAGEMENT

Why do we need a hierarchy to organize resources?
- ACCESS CONTROL : deciding who can view/edit/delete what (permissions)

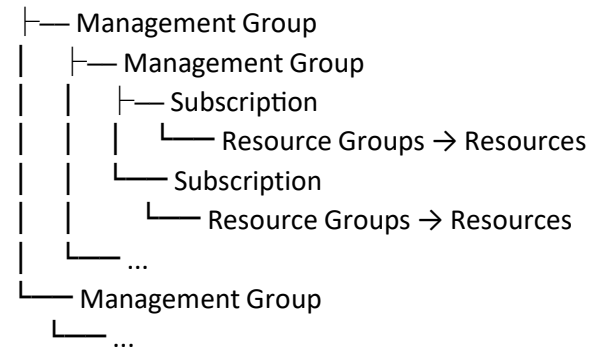
- POLICY CONTROL : enforcing rules, e.g. "no VM larger than a certain size", "only allow resources in India region"

Azure Resource Hierarchy (top to bottom):

Management Group (MG)
|
Subscription
|
Resource Group (RG)
|
Resource (VM, LB, IP, Disk, Snapshot)

[Root = the top-most Management Group, one per Azure AD directory/tenant]

Visual structure (from Azure docs):



[Management Group (MG) = a container used to group multiple subscriptions together so you can apply access/ policy rules to all of them at once instead of one by one]
[Subscription = a billing + access boundary; it's the unit that gets billed, and also the unit that separates environments, e.g. Dev subscription vs Prod subscription]
[Resource Group (RG) = a logical folder inside a subscription that groups related resources together, e.g. all resources for one application]

IMPORTANT RULE:
MG is the ONLY resource entity that can have another MG as its parent OR as its child — meaning Management Groups can be nested inside other Management Groups. Subscriptions, Resource Groups, and Resources CANNOT be nested like this.

5. SUBSCRIPTIONS & BILLING MODELS

Before creating resources, always think about PREDICTIVE COSTING [estimating cost in advance before you deploy

anything].

[Pricing Calculator = a free tool provided by every cloud provider (Azure, AWS, GCP) where you select the resources/ specs you plan to use, and it estimates your monthly bill BEFORE you actually create anything]

Common billing models:
---

a) PAY-AS-YOU-GO (PAYG)
- You pay only for what you use, billed periodically (usually monthly), no fixed upfront commitment.

b) FREE TRIAL (for individuals)
- You get a prepaid "wallet" linked to a temporary billing account.
- The wallet has an EXPIRY (time limit + credit limit).
  Example: Azure = 30 days AND $200 USD
  GCP = 90 days (3 months) AND $300 USD
- Your credit card is taken only to VERIFY YOUR IDENTITY at signup — it is NOT linked/charged to your billing account on day one.

[Trial expiry scenarios]:
1) If you use up all your credit in less than 30 days
   -> the wallet/trial becomes inactive immediately (even though days are still remaining).
2) If 30 days pass but you still have unused credit left
   -> the wallet/trial still becomes inactive (time limit wins, unused money doesn't roll over).

- Even after the free trial ends, you will NOT be charged a single penny UNLESS you separately add a Pay-As-You-Go subscription/billing account. You are allowed to add this PAYG billing account on day one itself, running in parallel with your free trial, so that once trial ends, billing continues seamlessly.

c) TIER-BASED / THRESHOLD MODEL
- A "tier" = a usage threshold. As long as your usage stays below that threshold, you are NOT charged, even without a formal trial period.

[Example: "always free tier" services — small VM sizes, limited storage, etc., that remain free indefinitely as long as you don't cross the limit]

d) AWS Billing Note:
- On AWS, from Day 1, your account is an ACTIVE POSTPAID billing account [postpaid = you use first, get billed afterward, like a mobile phone postpaid plan — unlike Azure/GCP's prepaid trial wallet model].

6. HOW DOES CHARGING ACTUALLY HAPPEN IN THE CLOUD?

[General concept — to be expanded with more detail as you learn]:
- Cloud providers meter your resource usage continuously (e.g., per second/per hour for VM uptime, per GB for storage, per GB for data transfer/egress).
- This usage is logged against your subscription/billing account.
- At the end of your billing cycle, the provider generates an invoice based on metered usage x the price rate for each resource type (as shown in the Pricing Calculator).
- If you're on PAYG, this amount is auto-charged to your linked payment method. If you're on trial, it's deducted from your wallet balance until it hits zero or expires.

[FAQs & doubts — leave this section open, fill in questions as they come up in class discussion]
-
-
-

==========
DAY 04 - SUBSCRIPTIONS, AZURE GEOGRAPHY & HIGH AVAILABILITY
==========

1. SUBSCRIPTION (Lab Recap)

A Subscription = a billing account resource that is linked to a payment method (credit card / bank account etc).
- You give it a NAME so it's identifiable, e.g. "Company Production"
- You can set a BUDGET, e.g. 500M USD, and Azure will track spend against it.

[Cost Analysis = an Azure tool (under Cost Management) that shows you a breakdown of how much you're spending, on which resource, over time — used to track and control cloud spend]

2. AZURE GEOGRAPHY (Physical Structure of the Cloud)

Azure's global infrastructure is organized in layers, biggest to smallest:

```
Geography (e.g. "India", "United States")
    |
Region              [a large geographical area]
    |
Availability Zone(AZ) [physically separate datacenter(s)
```

inside that region]
|
Datacenter [the actual physical building full of servers]

a) REGION
- A defined geographical area where Azure has deployed its infrastructure.
- Example: "Central India" is a region.

b) AVAILABILITY ZONE (AZ)
- A physically separate datacenter WITHIN a region, with its own independent power, cooling, and networking.
- Azure provides a MAXIMUM of 3 Availability Zones in any single region (this is a hard limit — some regions may have fewer or none).

Example - Central India [Region]
AZ 1
AZ 2
AZ 3

c) [Analogy example using NCR - National Capital Region]:
NCR = Region
- Delhi = Datacenter
- Noida = Datacenter
- Gurugram = Datacenter

[This is just an analogy to understand the concept — NCR itself is not an actual Azure region, it's used here to explain how one region can contain multiple physical datacenter locations]

d) DATACENTER
- The physical building containing racks of servers.

[Rack-mounted server = physical servers stacked in a metal frame/cabinet ("rack") inside the datacenter, e.g. 2U server, 4U server — "U" is a unit of rack height, 2U/4U tells you how much rack space that server occupies]

e) VIRTUAL DATACENTER (Azure's abstraction)
- What YOU experience in Azure is a "virtual" version of all this — you don't see physical racks, you just pick a Region/AZ from a dropdown, and Azure handles the physical placement behind the scenes.

[Reference tool]: datacenter.microsoft.com/globe/explore — Microsoft's interactive map of their global infrastructure:
- Square = Country name
- Circle = Region
- (zoom further) = Availability Zones

3. HIGH AVAILABILITY (HA) vs REDUNDANCY

These two terms are related but different:

- HIGH AVAILABILITY (a STATE)
  [Definition: the system is designed to remain accessible and running with minimal downtime — it's the GOAL/STATE you want to achieve]

- REDUNDANCY (a way to ACHIEVE that state)
  [Definition: having duplicate/backup copies of resources (servers, VMs, datacenters) so that if one fails, another takes over automatically — this is the METHOD used to achieve High Availability]

  In short: Redundancy is HOW you get High Availability.

[ACTIVE/PASSIVE (Clustering) explained]:
- A "cluster" = a group of servers working together as one system.
- ACTIVE/PASSIVE = one server (Active) handles all live traffic, while a second server (Passive) sits on standby, ready to take over instantly if the Active one fails. The Passive one does no work until it's needed.

4. REDUNDANCY LEVELS (from diagram) & SLA UPTIME

[SLA = Service Level Agreement — the % uptime guarantee a cloud provider promises you]

a) STANDARD REDUNDANCY (Single DC)
  - Your VM/resource exists in only ONE datacenter, inside one AZ.
  - Lower resilience — if that one datacenter has an issue, your app goes down.
  - SLA example: 99.90% uptime

b) ZONAL REDUNDANCY (1 VM in each DC/AZ)
  - You deploy ONE VM copy in EACH Availability Zone (multiple datacenters within the same region).
  - If one AZ/datacenter fails, traffic shifts to a VM running in another AZ — much higher resilience.
  - SLA example: 99.99% uptime

  [Why more 9s matter]: 99.90% uptime allows ~8.76 hours of downtime/year, while 99.99% allows only ~52 minutes/year — each extra "9" massively reduces allowed downtime.

--> SINGLE VM (Standard/Single DC)
  - Just one VM, no redundancy setup
  - SLA: 99.90%

--> AVAILABILITY SET
[Availability Set = a logical grouping that spreads multiple VMs across different physical racks/hardware WITHIN THE SAME datacenter — protects against hardware failure (a rack losing power, a host failing), but NOT against a whole datacenter going down, since it's all still in one DC]
- SLA: 99.95%

--> AVAILABILITY ZONE (Zonal)
- VMs spread across different AZs (different datacenters within the region), as in your diagram
- SLA: 99.99%

5. DIAGRAM WALKTHROUGH - LOAD BALANCED, ZONE-REDUNDANT SETUP

Structure shown:

User --> LB (Load Balancer) --> splits traffic to:
AZ 1 (DC) AZ 2 (DC)
[Server] [Server+VM] [Server+VM] [Server]

All of this sits inside one REGION / location.

- LB (Load Balancer) sits in front, receiving all user traffic and distributing it across multiple servers/AZs so no single server is overloaded, and if one AZ goes down, the LB routes traffic to the surviving AZ automatically.
- Each blue box (DC/AZ) contains multiple physical servers; some run a VM (shown as the small yellow "Vm" tag) actively serving the application.
- This whole setup = an example of ZONAL REDUNDANCY, since VMs are spread across 2 different AZs within the same region.

6. STATELESS vs STATEFUL APPLICATIONS

This distinction matters a lot for how you design redundancy/HA:

a) STATELESS APPLICATION
[Definition: the application does NOT store any session/user-specific data on the server itself between requests — every request is handled independently]
- Easy to make redundant: any server can respond to any request, since no server "remembers" anything unique.
- Example: a simple website that just serves static pages/API responses.

b) STATEFUL APPLICATION
[Definition: the application DOES store session/user-specific data on the server (e.g. login session, cart items, in-memory data) — so the request needs to keep going back to the SAME server, or that data needs to be shared/synced across servers]
- Harder to make redundant: needs "sticky sessions" (always route the same user to the same server) or a shared external state store (e.g. a database/cache), otherwise failover loses the user's data.
- Example: an old-school app storing your shopping cart in the server's local memory instead of a database.

[This is why databases (stateful) are much trickier to make highly available than web/app servers (often stateless) — you'll dig deeper into this when you study database replication later in the course]

7. DISASTER RECOVERY (DR) & REGION PAIRS

[Disaster Recovery (DR) = the plan/process to recover your application and data if an ENTIRE region goes down (e.g. natural disaster, major outage) — this goes one level beyond AZ redundancy]

[Region Pair = Azure pairs up two regions (usually within the same geography, e.g. two regions in India) so that if one entire region fails, your workload/data can fail over to its paired region]

AZ redundancy = protects against 1 datacenter failing
Region pair/DR = protects against an ENTIRE region failing

==========
DAY 04 (CONTINUED) - AVAILABILITY SETS, FAULT DOMAINS, RESOURCE SCOPE & LOCKS
==========

1. AVAILABILITY SET - DEEPER LOOK

Recap: Availability Set = redundancy WITHIN a single DC/AZ (as opposed to Zonal redundancy, which spreads VMs across MULTIPLE DCs/AZs).

WHAT IS A "SET"?
[SET = a combination of Racks + Servers that Azure uses to intelligently spread your VMs, so they don't all sit on the same physical hardware inside one datacenter]

Example: "3 rack, 5 server"
-> means Azure will spread your VM copies across 3 different racks, using up to 5 servers total, so that even if one rack loses power/network, your other VM copies (on other racks) stay alive.

Diagram walkthrough:
User --> Azure Orchestration --> distributes VMs across multiple RACKS inside one Availability Zone/Datacenter.

Each blue column = 1 RACK, containing multiple physical Servers stacked in it.
- Some servers run a VM (yellow "VM" tag) — these are your actual active VM copies.
- Notice in the diagram: VM copies are placed in DIFFERENT racks (Rack 1, Rack 3, Rack 4) — NOT in the same rack — this is the whole point of an Availability Set: spreading VM copies across different racks so a single rack failure doesn't take down all copies.

All of this happens WITHIN one Availability Zone/ Datacenter (bottom label: "Availability Zone || Datacenter") — this is what makes it different from Zonal redundancy, which spans MULTIPLE datacenters.

2. FAULT DOMAIN (FD) & FAULT TOLERANCE (FT)

- FT = FAULT TOLERANCE
[Definition: the ability of a system to keep working correctly even when part of it fails — this is the GOAL]

- FD = FAULT DOMAIN (labeled in diagram as "Rack")
[Definition: a group of hardware (like one rack) that shares a single point of failure — e.g. one power source, one network switch. If you put all your VM copies in the SAME fault domain/rack, a single power/switch failure kills ALL of them at once]

In short:
- Fault Domain = the "blast radius" grouping (1 rack = 1 fault domain, since it shares power/network)
- Fault Tolerance = the outcome you get by spreading VMs across MULTIPLE fault domains (racks), so one rack's failure doesn't wipe out your whole app.

An Availability Set = a way of using multiple Fault Domains (racks) to achieve Fault Tolerance within one DC.

3. HOW VM MOVEMENT WORKS UNDER THE HOOD (Supporting Concepts)

[HYPERVISOR = the software layer running on a physical server that allows it to host and run multiple Virtual Machines on top of the same physical hardware — think of it as the "landlord" software managing multiple VM "tenants" on one physical machine]

[vMotion = a live VM migration technology (originally a VMware term, similar concepts exist in Azure/Hyper-V) that moves a running VM from one physical host to another WITHOUT shutting it down or the user noticing — used during maintenance or hardware failure to keep the VM alive]

[IN-MEMORY COPY = while migrating a live VM (via vMotion-like tech), the VM's current RAM/memory state is copied over to the new host in real-time, so the VM can resume exactly where it left off on the new hardware, with no data loss or reboot]

These 3 concepts together explain HOW Azure can quietly move your VM to healthy hardware if a rack/server starts failing, without you noticing any downtime.

4. RESOURCE GROUP - CLEAR DEFINITION

[Resource Group (RG) = a logical container/folder name under which you can create and group MULTIPLE different resources together (VMs, disks, load balancers, etc.) — used purely for organization, access control, and lifecycle management (e.g. delete the whole RG to delete everything inside it at once)]

Example: A "Resource Group" named "WebApp-Prod-RG" could contain: 1 VM, 1 Disk, 1 Load Balancer, 1 Virtual Network — all belonging to the same application, managed together.

5. RESOURCE HIERARCHY BY SCOPE (Global / Regional / Zonal)

Every resource entity in Azure has a "scope" — i.e., HOW WIDE an area it applies to. This is a different lens on the same hierarchy you learned in Day-03:

a) GLOBAL RESOURCES (apply across all regions)
- Management Group (MG)
- Subscription

[These aren't tied to any one location — a Subscription or MG isn't "sitting" in a specific region, it's an umbrella covering resources anywhere]

b) REGIONAL RESOURCES (tied to a specific Region)
- Resource
- Resource Group

[When you create a Resource Group or most resources, you must PICK a region — that resource physically/logically lives in that region]

c) ZONAL RESOURCES (tied to a specific Availability Zone within a region)
- Certain resource types can be "pinned" to a specific AZ (e.g. a Zonal VM, Zonal Disk) rather than just a region generally.

[Why this matters]: Knowing the scope tells you where a setting/permission/policy actually applies — e.g. a Policy set at MG level cascades down to every Subscription/RG/Resource under it, while a Zonal setting only affects that one AZ.

6. RESOURCE LOCKS

[Lock = a resource protection mechanism in Azure that prevents accidental modification or deletion of a resource, REGARDLESS of what access permissions (RBAC) a user has — even an Owner/Admin can't bypass a lock without first removing it]

Two types of locks:

a) READ-ONLY LOCK
- Blocks BOTH modification AND deletion.
- Typical use case: PRODUCTION environment
  [where you want zero accidental changes or deletions — maximum protection, since Prod outages are costly]

b) DELETE LOCK
- Blocks ONLY deletion (modifications/edits are still allowed).
- Typical use case: UAT environment (User Acceptance Testing)
  [where testers/devs still need to modify/configure resources for testing, but you don't want someone accidentally deleting the whole environment]

Lock Type	Blocks Modify	Blocks Delete	Env
Read-only	Yes	Yes	Prod
Delete	No	Yes	UAT

[Note]: Locks work independently of RBAC (role-based access control) — they're an extra safety net layer on top of permissions, specifically to prevent "oops" mistakes.